Annexure A: Constraint-Based Problem Solving

a) PEAS Description

Component	Description
Performance measure	Number of constraint violations (target: zero clashes between conflicting courses); all 5 exams assigned to a slot; number of slots used (at most 3); computation time / search effort; optionally fairness (balanced load per day, gaps for students).
Environment	Set of 5 courses, 3 exam slots (one per day), the student-enrolment overlap data and the 7 conflict pairs (A-B, A-C, B-C, B-D, C-D, C-E, D-E). Static, fully observable, deterministic, discrete, single-agent.
Actuators	Assigning / reassigning a course to a slot, publishing the timetable, flagging unsolvable conflicts, notifying students and faculty.
Sensors	Enrolment database (shared-student information), list of available slots, current partial timetable, constraint-violation checker, new registration or slot-change inputs.

Input:

```
print("PEAS Description")

print("Performance Measure: Conflict-free exam timetable")
print("Environment: 5 courses and 3 exam slots")
print("Actuators: Assign courses to slots")
print("Sensors: Check courses and conflict constraints")
```

Ln: 6 Col: 56

Output:

```
    ========== RESTART: /Users/anushareddyyarramalla/Documents/peas discriotion 1.py ==========
    PEAS Description
    Performance Measure: Conflict-free exam timetable
    Environment: 5 courses and 3 exam slots
    Actuators: Assign courses to slots
    Sensors: Check courses and conflict constraints
>>>
    ========== RESTART: /Users/anushareddyyarramalla/Documents/peas discripion 11.py ==========
    PEAS Description
    Performance Measure: Conflict-free exam timetable
    Environment: 5 courses and 3 exam slots
    Actuators: Assign courses to slots
    Sensors: Check courses and conflict constraints
>>>
```

b) Type of Agent

Best suited: goal-based agent (with a utility-based extension only if soft preferences are added).

Clear goal state. The goal is explicit and binary: a complete assignment of all courses to slots with no conflicting pair sharing a slot.

Reflex agents fail. A simple reflex agent reacts only to the current percept and cannot plan a sequence of assignments; one wrong early choice may make the later ones impossible.

Beyond model-based. A model-based agent keeps state but still has no goal, so it does not know when it is done. The goal-based agent keeps the partial timetable as its model and searches (backtracking, forward checking) toward the goal.

Not utility-based. All that matters here is satisfiability of hard constraints. There is no numeric preference (such as minimising gaps) that would require utility maximisation, so a utility-based agent is unnecessary.

Input:

```
goal = "Schedule all exams without conflicts"

print("Agent Type: Goal-Based Agent")
print("Goal:", goal)
print("The agent searches for a valid timetable.")
```

Ln: 5 Col: 50

Output:

```
============================================================
Agent Type: Goal-Based Agent
Goal: Schedule all exams without conflicts
The agent searches for a valid timetable.
>>>
```



c) CSP Formulation

Element	Definition
Variables	X = {A, B, C, D, E}, one per course (the slot of that exam).
Domains	D(A) = D(B) = D(C) = D(D) = D(E) = {Slot1, Slot2, Slot3}
Constraints	Binary inequality constraints: A != B, A != C, B != C, B != D, C != D, C != E, D != E (each pair must receive different slots).

Constraint graph

Vertices: A, B, C, D, E (5 vertices).

Edges: (A,B), (A,C), (B,C), (B,D), (C,D), (C,E), (D,E) (7 edges).

Vertex	Degree	Neighbours
A	2	B, C
B	3	A, C, D
C	4	A, B, D, E
D	3	B, C, E
E	2	C, D

The graph contains the triangles A-B-C, B-C-D and C-D-E, so at least 3 slots are required. The problem is exactly 3-colouring of this graph.

Input:

IDLE Shell 3.13.4 | *untitled*

```
courses = ["A", "B", "C", "D", "E"]

slots = ["Slot1", "Slot2", "Slot3"]

conflicts = [
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("B", "D"),
    ("C", "D"),
    ("C", "E"),
    ("D", "E")
]

print("Variables:", courses)
print("Domains:", slots)
print("Constraint Graph Vertices:", courses)
print("Constraint Graph Edges:")

for edge in conflicts:
    print(edge)
```

Ln: 21 Col: 15

Output:

IDLE Shell 3.13.4 | peas discription 3.py

```
Python 3.13.4 (v3.13.4:8a526ec7cbe, Jun  3 2025, 21:14:54) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: /Users/anushareddyyarramalla/Documents/peas discription 3.py ==========
Variables: ['A', 'B', 'C', 'D', 'E']
Domains: ['Slot1', 'Slot2', 'Slot3']
Constraint Graph Vertices: ['A', 'B', 'C', 'D', 'E']
Constraint Graph Edges:
('A', 'B')
('A', 'C')
('B', 'C')
('B', 'D')
('C', 'D')
('C', 'E')
('D', 'E')
>>>
```

Ln: 16 Col: 0

d) Plain Backtracking Search

Variable order: A, B, C, D, E. Value order: Slot1, Slot2, Slot3. A value is checked only against already-assigned neighbours.

Step	Variable	Value tried	Check against assigned neighbours	Result
1	A	Slot1	No assigned neighbours	Accept: A=S1
2	B	Slot1	A=S1 (conflict A-B)	Reject
3	B	Slot2	A=S1 differs	Accept: B=S2
4	C	Slot1	A=S1 (conflict A-C)	Reject
5	C	Slot2	B=S2 (conflict B-C)	Reject
6	C	Slot3	A=S1, B=S2 differ	Accept: C=S3
7	D	Slot1	B=S2, C=S3 differ	Accept: D=S1
8	E	Slot1	D=S1 (conflict D-E)	Reject
9	E	Slot2	C=S3, D=S1 differ	Accept: E=S2 (complete)

Shaded rows are rejected attempts. Every rejected value was replaced by the next value of the same variable. No variable ever had its whole domain exhausted, so there was no need to return to a previously assigned variable (0 backtracks to earlier variables).

Final timetable: A = Slot1, B = Slot2, C = Slot3, D = Slot1, E = Slot2.

Slot	Courses	Check
Slot1 (Day 1)	A (AI), D (OS)	A-D is not a conflict pair
Slot2 (Day 2)	B (DBS), E (SE)	B-E is not a conflict pair
Slot3 (Day 3)	C (CN)	-

All 7 constraints verified: A!=B (S1,S2), A!=C (S1,S3), B!=C (S2,S3), B!=D (S2,S1), C!=D (S3,S1), C!=E (S3,S2), D!=E (S1,S2).

Input:

IDLE Shell 3.13.4 | *peas discription d.py*

```python
courses = ["A", "B", "C", "D", "E"]
slots = ["Slot1", "Slot2", "Slot3"]

conflicts = [
    ("A", "B"), ("A", "C"), ("B", "C"),
    ("B", "D"), ("C", "D"), ("C", "E"), ("D", "E")
]

assignment = {}
attempts = 0
backtracks = 0

def valid(course, slot):
    for x, y in conflicts:
        if x == course and y in assignment:
            if assignment[y] == slot:
                return False
        if y == course and x in assignment:
            if assignment[x] == slot:
                return False
    return True

def backtrack(index):
    global attempts, backtracks

    if index == len(courses):
        return True

    course = courses[index]

    for slot in slots:
        attempts += 1
        print("Trying", course, "=", slot)

        if valid(course, slot):
            assignment[course] = slot

            if backtrack(index + 1):
                return True

            del assignment[course]
            backtracks += 1
            print("Backtracking from", course)

        else:
            print("Conflict found!")

    return False

if backtrack(0):
    print("\nFinal Timetable:")
    for course in courses:
        print(course, ":", assignment[course])

print("Assignment attempts:", attempts)
print("Backtracks:", backtracks)
```

Ln: 56 Col: 32

output:

IDLE Shell 3.13.4 | peas discription 3.py

```
Python 3.13.4 (v3.13.4:8a526ec7cbe, Jun  3 2025, 21:14:54) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>> ========== RESTART: /Users/anushareddyyarramalla/Documents/peas discription 3.py ==========
Variables: ['A', 'B', 'C', 'D', 'E']
Domains: ['Slot1', 'Slot2', 'Slot3']
Constraint Graph Vertices: ['A', 'B', 'C', 'D', 'E']
Constraint Graph Edges:
('A', 'B')
('A', 'C')
('B', 'C')
('B', 'D')
('C', 'D')
('C', 'E')
('D', 'E')
>>>
```

Ln: 16 Col: 0

:

e) Backtracking with Forward Checking

After each assignment, the value is removed from the domain of every unassigned neighbour. If any domain becomes empty, the assignment is undone immediately. Same variable and value order as before.

Step	Assignment	A	B	C	D	E	Pruning / Remark
0	Initial	{1,2,3}	{1,2,3}	{1,2,3}	{1,2,3}	{1,2,3}	Full domains
1	A = Slot1	1	{2,3}	{2,3}	{1,2,3}	{1,2,3}	S1 removed from B, C (neighbours of A)
2	B = Slot2 (S1 no longer in D(B))	1	2	{3}	{1,3}	{1,2,3}	S2 removed from C, D. No empty domain
3	C = Slot3 (only value)	1	2	3	{1}	{1,2}	S3 removed from D, E
4	D = Slot1 (only value)	1	2	3	1	{2}	S1 removed from E
5	E = Slot2 (only value)	1	2	3	1	2	Complete, no wipe-out

In the table, 1, 2, 3 mean Slot1, Slot2, Slot3. Because Slot1 was pruned from B at step 1, B never even tries Slot1. Likewise C and D are left with a single legal value, so no rejected attempt occurs anywhere.

Final timetable: A = Slot1, B = Slot2, C = Slot3, D = Slot1, E = Slot2 (same as in part d)

Input:

IDLE Shell 3.13.4 | peas discription d.py | *untitled*

```
attempts = 0
backtracks = 0

def forward_check(course, slot):
    for x, y in conflicts:
        if x == course:
            neighbor = y
        elif y == course:
            neighbor = x
        else:
            continue

        if neighbor not in assignment:
            if slot in domains[neighbor]:
                domains[neighbor].remove(slot)

            if len(domains[neighbor]) == 0:
                return False

    return True

def backtrack(index):
    global attempts, backtracks

    if index == len(courses):
        return True

    course = courses[index]

    for slot in domains[course][:]:
        attempts += 1
        print("Trying", course, "=", slot)

        old_domains = {
            c: domains[c].copy()
            for c in courses
        }

        assignment[course] = slot
        domains[course] = [slot]

        if forward_check(course, slot):
            print("Domains after pruning:", {
                c: domains[c]
                for c in courses if c not in assignment
            })

            if backtrack(index + 1):
                return True

        assignment.pop(course)
        domains.update(old_domains)
        backtracks += 1
        print("Backtracking from", course)

    return False

if backtrack(0):
    print("\nFinal Timetable:")
    for course in courses:
        print(course, ":", assignment[course])

print("Assignment attempts:", attempts)
print("Backtracks:", backtracks)
```

Ln: 18 Col: 25

Output:

```
Trying A = Slot1
Domains after pruning: {'B': ['Slot2', 'Slot3'], 'C': ['Slot2', 'Slot3'], 'D': ['Slot1', 'Slot2', 'Slot3'], 'E': ['Slot1', 'Slot2', 'Slot3']}
Trying B = Slot2
Domains after pruning: {'C': ['Slot3'], 'D': ['Slot1', 'Slot3'], 'E': ['Slot1', 'Slot2', 'Slot3']}
Trying C = Slot3
Domains after pruning: {'D': ['Slot1'], 'E': ['Slot1', 'Slot2']}
Trying D = Slot1
Domains after pruning: {'E': ['Slot2']}
Trying E = Slot2
Domains after pruning: {}

Final Timetable:
A : Slot1
B : Slot2
C : Slot3
D : Slot1
E : Slot2
Assignment attempts: 5
Backtracks: 0
>>>
```

f) Comparison

Measure	Plain Backtracking	Backtracking + Forward Checking
Assignment attempts	9	5
Rejected (failed) attempts	4 (B=S1, C=S1, C=S2, E=S1)	0
Backtracks to earlier variables	0	0

Measure	Plain Backtracking	Backtracking + Forward Checking
Consistency work	Done at assignment time, after the value is picked	Done once per assignment by pruning neighbours
Early failure detection	No (conflict found only when the variable is reached)	Yes (empty domain detected immediately)
Solution found	A=S1, B=S2, C=S3, D=S1, E=S2	Same

Why forward checking reduces search effort

- **Prunes in advance.** Conflicting values are removed from neighbours' domains as soon as a variable is assigned, so they are never tried later (B=S1, C=S1, C=S2 and E=S1 were never attempted).
- **Early dead-end detection.** If a neighbour's domain becomes empty, the algorithm backtracks at once instead of discovering the dead end several levels deeper. That did not happen on this instance, but it is the main source of savings on harder ones.
- **Forced moves.** Variables whose domains shrink to one value (C, D, E here) are assigned directly with no trial and error.
- **Trade-off.** Forward checking costs extra work per assignment (updating neighbour domains), but this is small compared with the avoided failed attempts, so overall it is more efficient. Here it cut assignment attempts from 9 to 5 (about 44% fewer).

Input:

IDLE Shell 3.13.4 | peas discription d.py | peas 55.py | peas discription 5.py | *untitled*

```
print("Comparison of Search Algorithms")

plain_attempts = 9
plain_backtracks = 0

forward_attempts = 5
forward_backtracks = 0

print("\nPlain Backtracking")
print("Assignment Attempts:", plain_attempts)
print("Backtracks:", plain_backtracks)

print("\nForward Checking")
print("Assignment Attempts:", forward_attempts)
print("Backtracks:", forward_backtracks)

print("\nConclusion:")
print("Forward Checking requires fewer assignment attempts.")
```

Ln: 18 Col: 61

Output:

```
>>>
========== RESTART: /Users/anushareddyyarramalla/D
Comparison of Search Algorithms

Plain Backtracking
Assignment Attempts: 9
Backtracks: 0

Forward Checking
Assignment Attempts: 5
Backtracks: 0

Conclusion:
Forward Checking requires fewer assignment attempts.
>>>
```